Topic and Sentiment Evolution in a Politician's Policy Commentary

Kevin

2026-06-18

1 Purpose and analytic framing

This document specifies a reproducible pipeline for studying **how the policy topics and the sentiment in a politician's public commentary change over time**. The text-mining methods follow Silge & Robinson, *Text Mining with R: A Tidy Approach* (the tidytext book).

The full set of comments is in hand, so the corpus is analyzed as a **census**, not a sample: there is no inclusion-probability or selection-weighting step. The workflow's defensibility rests on demonstrating, against a known data-generating process, that the pipeline recovers structure it never got to see. Two error sources remain in play:

- **Measurement error.** A topic model and a sentiment score are error-prone *measurements*. Lexicon sentiment in particular is unigram-based and misreads negation ("not good"); the book is explicit about this, and we operationalize the caution as a diagnostic. Vocabulary and framing also drift over a decade, threatening measurement comparability (the textual analogue of measurement non-invariance).
- **Processing error.** Tokenization, stop-word lists, stemming, and chunk size are analyst choices that propagate downstream and should be recorded and sensitivity-checked.

The plan: (1) simulate a corpus with a **known** time-varying topic structure *and* a **known** sentiment (valence) structure; (2) run the tidytext pipeline (frequencies, tf-idf, topics, lexicon sentiment, negation diagnostic); (3) recover topic and sentiment trajectories and **validate them against ground truth**; (4) add an optional AI module using a **local** model (Gemma via Ollama through ellmer) for topic labeling and sentiment, cross-checked against the lexicon. Because the data-generating process is known, the workflow can demonstrate that it recovers truth before it is pointed at real data. If you later validate topics by hand-coding only a subsample rather than the whole corpus, draw that subsample as a probability sample with known selection probabilities — a separate two-phase weighting step, not included here.

> **i** Why simulate first
>
> On real found-text data the ground truth is unknown, so you cannot tell whether the pipeline is recovering structure or inventing it. Validating against a known data-generating process is the standard defense. The "Adapting this workflow to the real corpus" section lists exactly which lines to swap for the real corpus.

2 Reproducibility: package installation

This document renders to PDF through Quarto's built-in **Typst** engine, so no LaTeX distribution (TeX Live / tinytex) is required — `quarto render` compiles the PDF directly. It needs Quarto >= 1.4 (Typst is bundled). Figures use a PNG device because Typst embeds raster and SVG images but not PDF figures.

```
# Run once. {stm} pulls in {tm} and {SnowballC}.
install.packages(c(
  "tidyverse", "tidytext", "topicmodels", "stm", "tm", "SnowballC",
  "clue", "glue", "scales"
))

# Sentiment lexicons: "bing" ships with tidytext; "afinn"/"nrc" require {textdata}
# and a ONE-TIME interactive license acceptance. Run these once interactively:
install.packages("textdata")
# textdata::lexicon_afinn()   # accept license at the prompt
# textdata::lexicon_nrc()     # accept license at the prompt

# Optional AI module (Section 9): local LLMs via Ollama + ellmer.
install.packages(c("ellmer", "irr"))
#  - Install Ollama (https://ollama.com), then pull a local model, e.g.:
#       ollama pull gemma4        # or your local gemma/llama tag
#  - Confirm the exact local tag from R with ellmer::models_ollama().
```

3 Simulation design (the data-generating process)

All structural parameters live in one place so the simulation is fully specified and tunable.

```
dgp <- list(
  years          = 2014:2025,        # decade of commentary
  posts_per_year = 150L,             # "hundreds per year"; raise for realism
  K              = 6L,               # number of latent policy topics
  theta_noise_sd = 0.55,             # document-level dispersion of topic mix
  n_sent_range   = 3:6               # sentences per post (paragraph length)
)

topic_names <- c(
  "Economy & Taxation",
  "Healthcare",
  "Immigration & Borders",
  "Climate & Energy",
  "National Security & Defense",
  "Education"
)
```

3.1 Topic vocabularies and sentence templates

Each latent topic is a small bank of distinctive content words; overlap between banks is minimized so recovery is unambiguous (on real data, topics overlap and recovery is harder, a limitation noted in the Limitations section). Sentences are assembled from templates carrying realistic function-word noise so the downstream stop-word removal has something to do.

```r
topic_vocab <- list(
  c("economy","taxes","taxation","budget","deficit","inflation","wages",
    "growth","tariffs","investment","fiscal","revenue","spending","debt"),
  c("healthcare","hospitals","insurance","patients","doctors","nurses",
    "coverage","clinics","premiums","prescriptions","treatment","medicine"),
  c("immigration","border","migrants","asylum","visas","citizenship",
    "refugees","deportation","integration","quotas","residency","newcomers"),
  c("climate","emissions","renewables","solar","wind","carbon","pollution",
    "sustainability","fossil","electricity","grid","greenhouse"),
  c("defense","military","terrorism","intelligence","cyber","alliances",
    "sovereignty","troops","surveillance","deterrence","drones","missiles"),
  c("education","schools","students","teachers","universities","curriculum",
    "tuition","literacy","classrooms","scholarships","graduates","learning")
)
names(topic_vocab) <- topic_names

# Neutral templates (function-word noise + two topic content words).
templates <- c(
  "We must address {w1} and {w2} for families across the country.",
  "My position on {w1} is clear: the government should act on {w2}.",
  "Today I spoke in Parliament about {w1} and our {w2} policy.",
  "For too long we have debated {w1} and the question of {w2}.",
  "Our plan this session covers {w1} alongside {w2}.",
  "Citizens have raised concerns about {w1} and {w2}.",
  "This budget cycle we will turn to {w1} and {w2}.",
  "The national conversation on {w1} and {w2} continues."
)
```

3.2 Time-varying topic prevalence (ground truth)

Each topic gets an unnormalized intensity as a smooth function of the year index $t = 0, 1, \ldots$, mapped to a prevalence vector $\pi(t)$ via a softmax:

$$\pi_k(t) = \frac{\exp(\eta_k(t))}{\sum_{j=1}^{K} \exp\big(\eta_j(t)\big)}.$$

Planted trajectories: **Economy** declining, **Healthcare** and **Climate** rising, **Immigration** spiking mid-period, **Security** with a later bump, **Education** roughly flat. These are the curves the pipeline must recover.

```r
t_idx <- seq_along(dgp$years) - 1L   # 0..(T-1)

eta <- tibble(
  t        = t_idx,
  economy  = 2.0 - 0.16 * t,
  health   = 0.4 + 0.20 * t,
  immig    = 2.3 * exp(-0.5 * ((t - 3) / 1.6)^2),
  climate  = 0.1 + 0.24 * t,
  security = 0.6 + 1.1 * exp(-0.5 * ((t - 7) / 1.3)^2),
  educ     = 0.9 - 0.02 * t
```

```
)

softmax_rows <- function(m) {
  ex <- exp(m - apply(m, 1, max))
  ex / rowSums(ex)
}

pi_mat <- eta |>
  dplyr::select(-t) |>
  as.matrix() |>
  softmax_rows()
colnames(pi_mat) <- topic_names
rownames(pi_mat) <- dgp$years

truth_prevalence <- pi_mat |>
  as_tibble(rownames = "year") |>
  mutate(year = as.integer(year)) |>
  pivot_longer(-year, names_to = "topic", values_to = "true_prev")
```

```
truth_prevalence |>
  mutate(topic = factor(topic, levels = topic_names)) |>
  ggplot(aes(year, true_prev, color = topic)) +
  geom_line(linewidth = 0.9) +
  geom_point(size = 1.2) +
  scale_color_viridis_d(end = 0.92) +
  scale_x_continuous(breaks = dgp$years) +
  scale_y_continuous(labels = label_percent()) +
  labs(x = NULL, y = "Share of attention", color = NULL,
       title = "Ground-truth topic prevalence") +
  theme(legend.position = "bottom",
        axis.text.x = element_text(angle = 45, hjust = 1))
```

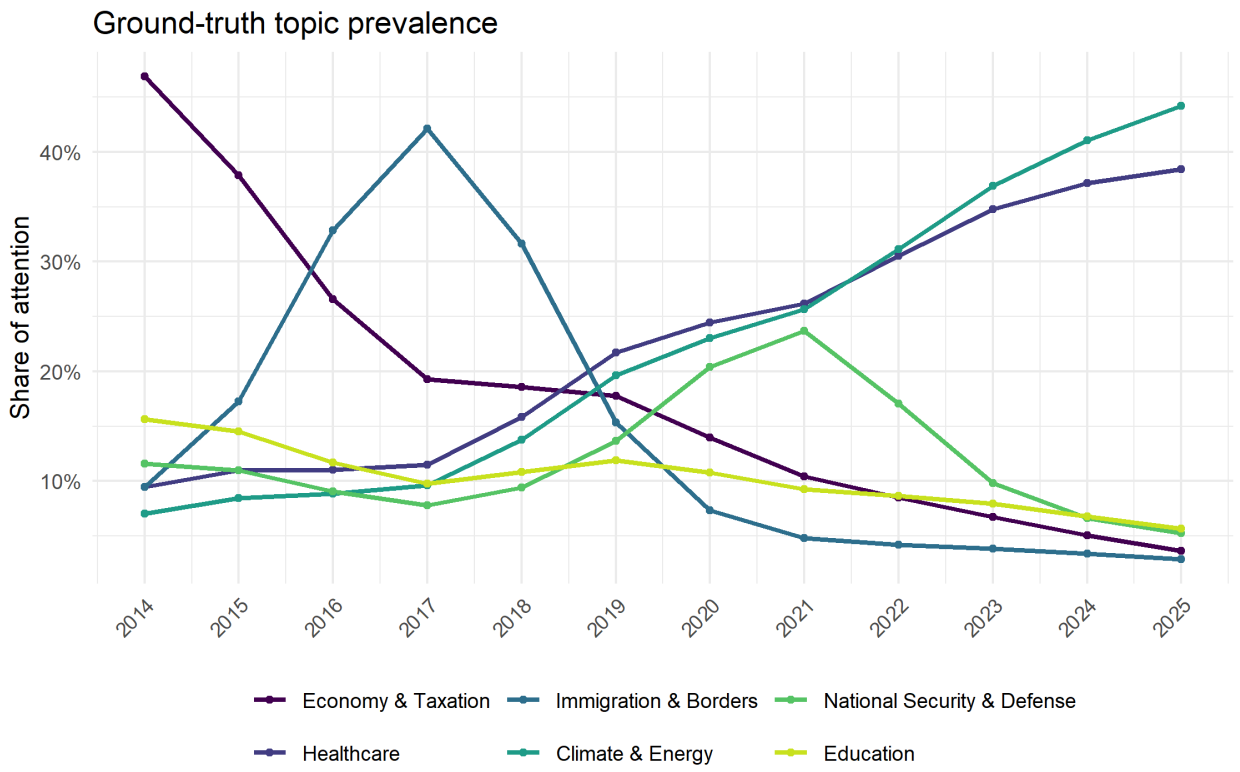


Figure 1: Planted (ground-truth) topic prevalence by year.

3.3 Time-varying sentiment / valence (ground truth)

Sentiment is planted separately from topic, and **each topic gets its own tone trajectory** — some souring, some improving, some flat — so the per-topic sentiment shape is a genuine, discriminating signal to recover rather than one common trend shared by every topic. For a sentence on topic k in year-index t, the latent valence is

$$m_k(t) = v_k + \delta_k\left(t - \bar{t}\right),$$

with intercept v_k and a **topic-specific** slope δ_k. The sentence is drawn positive / neutral / negative from a softmax in $m_k(t)$; positive sentences append a word from a positive bank, negative ones from a negative bank. Both banks are chosen to live in the Bing and AFINN lexicons so the lexical measurement can recover them.

```
# Valence words chosen to be present in the Bing lexicon (and mostly AFINN).
valence_words <- list(
  positive = c("support","benefit","strong","proud","secure","fair",
               "success","gain"),
  negative = c("fail","failed","fear","weak","corrupt","threat",
               "attack","crisis")
)

# Per-topic valence intercept v_k and slope delta_k (order = topic_names).
# Distinct slopes => distinct sentiment shapes, so recovery is a real test, not
```

```r
# a single common trend that z-scoring would erase.
topic_valence_intercept <- c(-0.10, 0.10, -0.50, 0.20, -0.55, 0.35)
topic_valence_slope     <- c(-0.14, 0.12, -0.06, 0.16,  0.00, 0.04)

sentence_valence_prob <- function(k, t) {
  m   <- topic_valence_intercept[k] + topic_valence_slope[k] * (t - mean(t_idx))
  a   <- 1.2
  raw <- c(negative = exp(-a * m), neutral = exp(0.2), positive = exp(a * m))
  raw / sum(raw)
}

make_sentence <- function(k, t) {
  ws   <- sample(topic_vocab[[k]], 2)
  base <- glue(sample(templates, 1), w1 = ws[1], w2 = ws[2])
  vp   <- sentence_valence_prob(k, t)
  val  <- sample(c("negative", "neutral", "positive"), 1, prob = vp)
  if (val == "neutral") return(as.character(base))
  vw <- sample(valence_words[[val]], sample(1:2, 1))
  paste(base, paste(vw, collapse = " "))
}

# Planted net valence (P(positive) - P(negative)) by topic and year = ground truth.
valence_truth <- expand_grid(topic_i = seq_len(dgp$K), t = t_idx) |>
  mutate(
    probs = map2(topic_i, t, sentence_valence_prob),
    net   = map_dbl(probs, \(p) p[["positive"]] - p[["negative"]]),
    topic = topic_names[topic_i],
    year  = t + min(dgp$years)
  )
```

```r
valence_truth |>
  mutate(topic = factor(topic, levels = topic_names)) |>
  ggplot(aes(year, net, color = topic)) +
  geom_line(linewidth = 0.9) +
  scale_color_viridis_d(end = 0.92) +
  scale_x_continuous(breaks = dgp$years) +
  geom_hline(yintercept = 0, linewidth = 0.3, linetype = "dotted") +
  labs(x = NULL, y = "Net valence (truth)", color = NULL,
       title = "Ground-truth sentiment by topic over time") +
  theme(legend.position = "bottom",
        axis.text.x = element_text(angle = 45, hjust = 1))
```

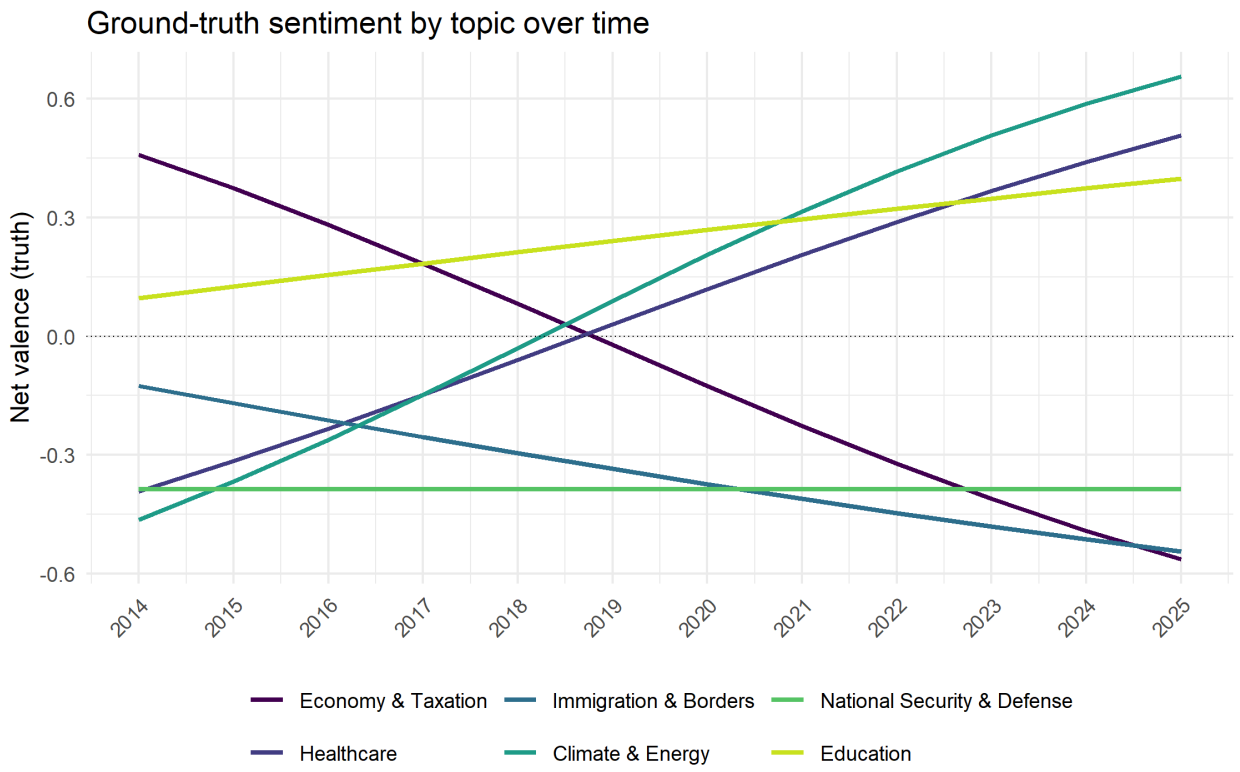


Figure 2: Planted net valence (P(positive) - P(negative)) by topic and year.

3.4 Generating the corpus

Each post's topic mixture θ_d perturbs the year's prevalence vector on the log scale; each sentence draws a topic from θ_d and a valence from the topic-and-year valence probabilities. Construction is vectorized with `purrr` (no `for`/`while` loops).

```
draw_theta <- function(t) {
  p   <- pi_mat[t + 1L, ]
  raw <- p * exp(rnorm(dgp$K, 0, dgp$theta_noise_sd))
  raw / sum(raw)
}

generate_post <- function(theta, t) {
  n_sent    <- sample(dgp$n_sent_range, 1)
  topic_seq <- sample(seq_len(dgp$K), n_sent, replace = TRUE, prob = theta)
  map_chr(topic_seq, \(k) make_sentence(k, t)) |> paste(collapse = " ")
}

set.seed(613)
corpus <- map_dfr(
  dgp$years,
  \(yr) tibble(year = yr, t = yr - min(dgp$years))
) |>
  group_by(year) |>
  slice(rep(1, dgp$posts_per_year)) |>
```

```
  ungroup() |>
  mutate(
    doc_id   = sprintf("post_%05d", row_number()),
    theta    = map(t, draw_theta),
    dominant = map_int(theta, which.max),
    dominant = factor(topic_names[dominant], levels = topic_names),
    text     = map2_chr(theta, t, generate_post)
  )

glue("Corpus (all comments): {nrow(corpus)} posts across ",
     "{min(dgp$years)}-{max(dgp$years)}.")
```

```
Corpus (all comments): 1800 posts across 2014-2025.
```

4 Text preprocessing with tidytext

From here the code is **identical to what it would be on real data**: it operates on `corpus$text`, never on the hidden truth columns. Tokenization uses one-token-per-row form, drops stop words via an anti-join, and removes numeric tokens, following the tidytext book.

```
tokens <- corpus |>
  dplyr::select(doc_id, year, text) |>
  unnest_tokens(word, text) |>
  anti_join(get_stopwords(), by = "word") |>
  filter(!str_detect(word, "\\d"))

glue("Vocabulary after cleaning: {n_distinct(tokens$word)} terms; ",
     "{nrow(tokens)} tokens.")
```

```
Vocabulary after cleaning: 118 terms; 53206 tokens.
```

⚠ Processing-error checkpoint

Stop-word lists, stemming, and minimum-frequency trimming are choices, not neutral defaults. Record them; report a sensitivity check in which headline results are re-estimated under at least one alternative recipe.

5 Exploratory text analysis: how emphasis shifts over time

A natural first look would be tf-idf with each *year* as a document,

$$\mathrm{tfidf}(w, y) = \mathrm{tf}(w, y) \cdot \ln \frac{N}{\mathrm{df}(w)},$$

with $\mathrm{tf}(w, y)$ the relative frequency of term w in year y (its share of that year's tokens), N the number of years, and $\mathrm{df}(w)$ the number of years containing w. On *this* corpus it is degenerate: the simulated author's vocabulary is stable across the decade, so every term appears in every year, $\mathrm{df}(w) = N$, the factor

$\ln(N/\mathrm{df})$ is zero for every term, and tf-idf collapses to zero everywhere — an all-zero plot. That is the correct answer here, and an instructive one: what changes over time in this corpus is not *which* words appear but *how often each topic is discussed*.

The direct view of that shift is the share of each year's tokens taken by a few topic-marker terms,

$$f(w, y) = \frac{n(w, y)}{\sum_{w'} n(w', y)},$$

which previews the prevalence trends the topic models quantify formally. (On real text, where vocabulary genuinely turns over year to year, the tf-idf view above is informative and worth keeping.)

```
markers <- c("taxes", "insurance", "border", "emissions", "military", "schools")

tokens |>
  filter(word %in% markers) |>
  dplyr::count(year, word) |>
  left_join(dplyr::count(tokens, year, name = "year_total"), by = "year") |>
  mutate(share = n / year_total) |>
  ggplot(aes(year, share, color = word)) +
  geom_line(linewidth = 0.9) +
  geom_point(size = 1) +
  scale_color_viridis_d(end = 0.92) +
  scale_x_continuous(breaks = dgp$years) +
  labs(x = NULL, y = "Share of yearly tokens", color = "Topic marker",
       title = "Topic-marker term frequency over time") +
  theme(legend.position = "bottom",
        axis.text.x = element_text(angle = 45, hjust = 1))
```

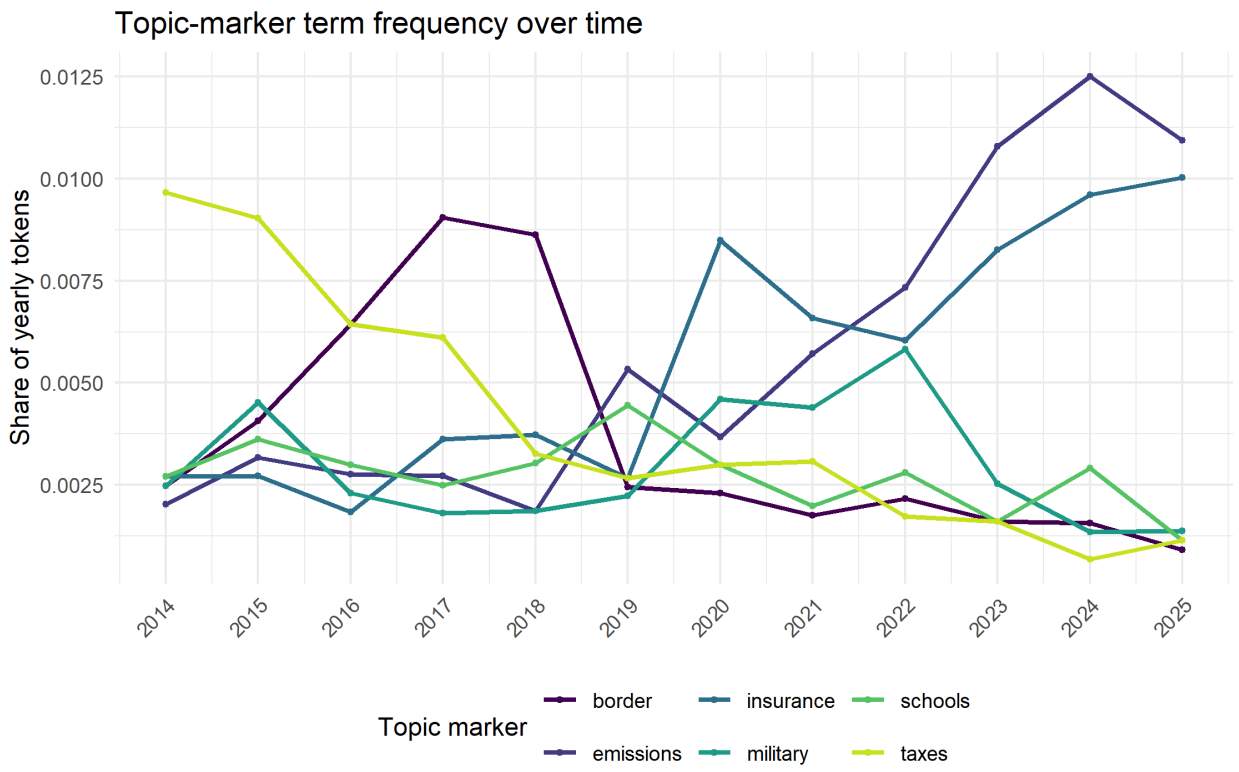


Figure 3: Share of each year's tokens taken by one marker term per topic — a model-free preview of the planted prevalence shifts.

6 Topic model 1: Latent Dirichlet Allocation (book Ch. 6)

LDA's generative story is

$$\theta_d \sim \mathrm{Dir}(\alpha), \quad z_{dn} \sim \mathrm{Cat}(\theta_d), \quad w_{dn} \sim \mathrm{Cat}\left(\beta_{z_{dn}}\right),$$

with θ_d the document-topic mixture (gamma) and β_k the topic-word distribution (beta). **LDA has no notion of time**; we estimate per-document topic shares and aggregate by year afterward.

6.1 Choosing the number of topics

K is the key tuning parameter. The documented procedure is a held-out perplexity sweep plus interpretability review; it is shown but not executed in the render. We proceed with $K = 6$ because the simulation planted six topics; on real data, **choose** K via this procedure, not by assumption.

```
dtm_all <- corpus |> unnest_tokens(word, text) |>
  anti_join(get_stopwords(), by = "word") |>
  filter(!str_detect(word, "\\d")) |>
  dplyr::count(doc_id, word) |>
  cast_dtm(doc_id, word, n)

map_dfr(4:9, function(k) {
  fit <- LDA(dtm_all, k = k, method = "Gibbs",
             control = list(seed = 1, iter = 800, burnin = 200))
```

```
  tibble(k = k, perplexity = perplexity(fit, dtm_all))
}) |>
  ggplot(aes(k, perplexity)) + geom_line() + geom_point() +
  labs(title = "Held-out perplexity by K (lower = better)")
```

6.2 Fitting LDA and extracting topics

```
word_counts <- tokens |> dplyr::count(doc_id, word)
dtm <- word_counts |> cast_dtm(doc_id, word, n)

lda_fit <- LDA(
  dtm, k = dgp$K, method = "Gibbs",
  control = list(seed = 1234, iter = 1500, burnin = 300, alpha = 0.1)
)

lda_beta  <- tidy(lda_fit, matrix = "beta")
lda_gamma <- tidy(lda_fit, matrix = "gamma")
```

```
lda_beta |>
  group_by(topic) |>
  slice_max(beta, n = 8, with_ties = FALSE) |>
  ungroup() |>
  mutate(term = reorder_within(term, beta, topic)) |>
  ggplot(aes(beta, term, fill = factor(topic))) +
  geom_col(show.legend = FALSE) +
  scale_fill_viridis_d() +
  scale_y_reordered() +
  facet_wrap(~ topic, scales = "free_y") +
  labs(x = expression(beta), y = NULL, title = "LDA topic-term distributions")
```

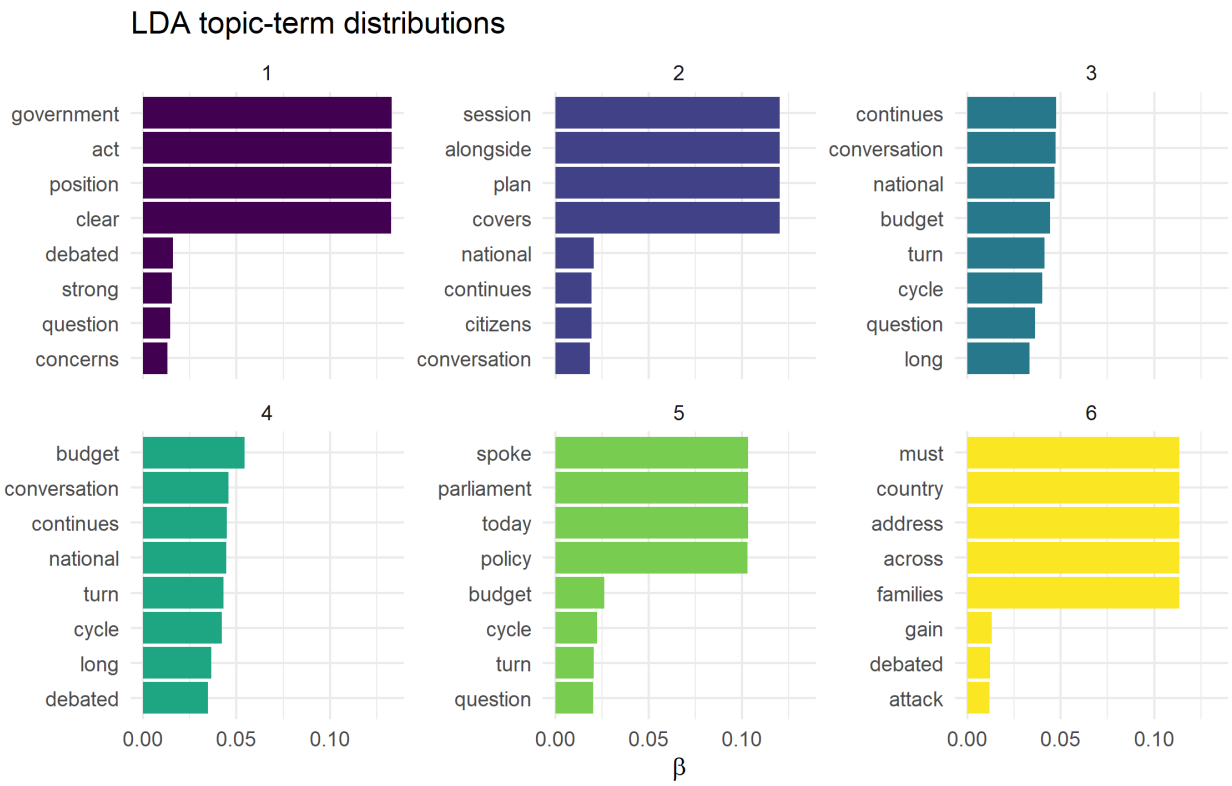


Figure 4: Top terms per LDA topic (numbered arbitrarily until aligned in 7.3).

6.3 Aligning estimated topics to ground truth (Hungarian algorithm)

Topic models return topics in arbitrary order. We align recovered topics to the planted ones by solving a linear assignment problem (Hungarian algorithm) on a term-overlap cost matrix.

```
vocab_lookup <- imap_dfr(topic_vocab, \(words, nm) tibble(term = words, planted = nm))

align_topics <- function(top_terms_df) {
  # Overlap count of each estimated topic's top terms with each planted vocab.
  ov_long <- top_terms_df |>
    left_join(vocab_lookup, by = "term") |>
    filter(!is.na(planted)) |>
    dplyr::count(topic, planted)

  # Complete the topic x planted grid so the cost matrix is always square (K x K),
  # even for a topic whose top terms overlap no planted vocabulary (count 0).
  # Without this, solve_LSAP() on a non-square matrix leaves a planted topic
  # unmapped, and any later lookup of that topic returns integer(0).
  all_topics <- sort(unique(top_terms_df$topic))
  ov <- expand_grid(topic = all_topics, planted = topic_names) |>
    left_join(ov_long, by = c("topic", "planted")) |>
    mutate(n = coalesce(n, 0L)) |>
    pivot_wider(names_from = planted, values_from = n, values_fill = 0L) |>
    column_to_rownames("topic")
  ov <- ov[, topic_names, drop = FALSE]
```

```
  # Hungarian assignment: maximize overlap (minimize max-overlap minus overlap).
  a <- solve_LSAP(max(as.matrix(ov)) - as.matrix(ov))
  tibble(topic = as.integer(rownames(ov)),
         planted_name = colnames(ov)[as.integer(a)])
}

lda_topic_map <- lda_beta |>
  group_by(topic) |>
  slice_max(beta, n = 10, with_ties = FALSE) |>
  ungroup() |>
  rename(term = term) |>
  align_topics()
lda_topic_map
```

```
# A tibble: 6 × 2
  topic planted_name
  <int> <chr>
1     1 Healthcare
2     2 Immigration & Borders
3     3 Economy & Taxation
4     4 National Security & Defense
5     5 Education
6     6 Climate & Energy
```

6.4 LDA-based prevalence over time and validation

```
lda_prev <- lda_gamma |>
  rename(doc_id = document) |>
  mutate(topic = as.integer(topic)) |>
  left_join(lda_topic_map, by = "topic") |>
  left_join(dplyr::select(corpus, doc_id, year), by = "doc_id") |>
  group_by(year, planted_name) |>
  summarise(est_prev = mean(gamma), .groups = "drop")

lda_prev |>
  ggplot(aes(year, est_prev, color = planted_name)) +
  geom_line(linewidth = 0.9) +
  geom_line(data = truth_prevalence |> rename(planted_name = topic),
            aes(year, true_prev), linetype = "dashed", linewidth = 0.5) +
  scale_color_viridis_d(end = 0.92) +
  scale_x_continuous(breaks = dgp$years) +
  scale_y_continuous(labels = label_percent()) +
  facet_wrap(~ planted_name) +
  labs(x = NULL, y = "Share of attention",
       title = "LDA recovery vs. ground truth") +
  theme(legend.position = "none",
        axis.text.x = element_text(angle = 45, hjust = 1, size = 6))
```

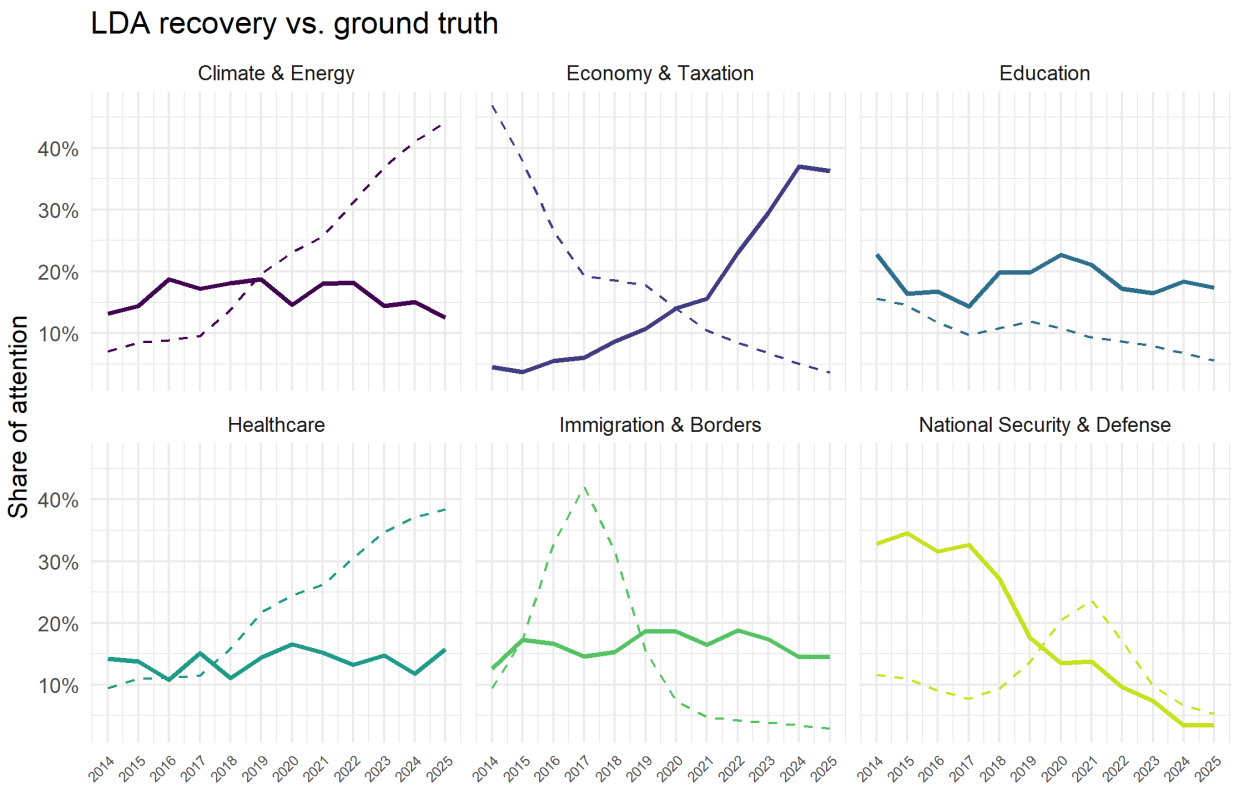


Figure 5: LDA-recovered prevalence (lines) vs. ground truth (dashed).

```
lda_prev |>
  left_join(truth_prevalence, by = c("year", "planted_name" = "topic")) |>
  group_by(planted_name) |>
  summarise(correlation = cor(est_prev, true_prev), .groups = "drop")
```

```
# A tibble: 6 × 2
  planted_name                correlation
  <chr>                             <dbl>
1 Climate & Energy                 -0.308
2 Economy & Taxation               -0.803
3 Education                         0.326
4 Healthcare                        0.236
5 Immigration & Borders            -0.168
6 National Security & Defense      -0.110
```

7 Topic model 2: Structural Topic Model (time-aware, recommended)

LDA's post-hoc aggregation has no principled standard error on a trend. The **Structural Topic Model** lets topic prevalence depend on covariates; here, smoothly on year:

$$\theta_d \sim \text{LogisticNormal}(\Gamma' x_d, \Sigma), \quad x_d = \left[1, s\left(\text{year}_d\right)\right].$$

This is the cleaner tool for "how do topics evolve over time," estimating the time effect jointly with the topics. STM extends beyond the SURV727 core toolkit. One processing wrinkle to record: `textProcessor()` applies tm's stop-word list and its own tokenizer, which differs slightly from the tidytext path above (`get_stopwords()`), so the LDA and STM vocabularies are close but not identical — a small processing-error source, not a bug.

```
processed <- textProcessor(
  documents = corpus$text,
  metadata  = corpus |> dplyr::select(doc_id, year),
  lowercase = TRUE, removestopwords = TRUE, removenumbers = TRUE,
  removepunctuation = TRUE, stem = FALSE, verbose = FALSE
)
prepped <- prepDocuments(processed$documents, processed$vocab,
                         processed$meta, lower.thresh = 5, verbose = FALSE)

stm_fit <- stm(
  documents = prepped$documents, vocab = prepped$vocab, K = dgp$K,
  prevalence = ~ s(year), data = prepped$meta,
  init.type = "Spectral", seed = 740, verbose = FALSE
)
```

```
stm_terms <- labelTopics(stm_fit, n = 8)
stm_top <- imap_dfr(seq_len(dgp$K),
                    \(k, .) tibble(topic = k, term = stm_terms$frex[k, ]))
stm_topic_map <- align_topics(stm_top)
stm_topic_map
```

```
# A tibble: 6 × 2
  topic planted_name
  <int> <chr>
1     1 Education
2     2 Economy & Taxation
3     3 Climate & Energy
4     4 National Security & Defense
5     5 Healthcare
6     6 Immigration & Borders
```

```
# Formal model-based test of the time effect (with uncertainty).
stm_effect <- estimateEffect(1:dgp$K ~ s(year), stm_fit,
                             metadata = prepped$meta, uncertainty = "Global")
```

```
stm_gamma <- stm_fit$theta |>
  as_tibble(.name_repair = ~ as.character(seq_len(dgp$K))) |>
  mutate(doc_id = prepped$meta$doc_id, year = prepped$meta$year) |>
  pivot_longer(c(-doc_id, -year), names_to = "topic", values_to = "gamma") |>
  mutate(topic = as.integer(topic)) |>
  left_join(stm_topic_map, by = "topic")
```

```r
stm_prev <- stm_gamma |>
  group_by(year, planted_name) |>
  summarise(est_prev = mean(gamma), .groups = "drop")

stm_prev |>
  ggplot(aes(year, est_prev, color = planted_name)) +
  geom_line(linewidth = 0.9) +
  geom_line(data = truth_prevalence |> rename(planted_name = topic),
            aes(year, true_prev), linetype = "dashed", linewidth = 0.5) +
  scale_color_viridis_d(end = 0.92) +
  scale_x_continuous(breaks = dgp$years) +
  scale_y_continuous(labels = label_percent()) +
  facet_wrap(~ planted_name) +
  labs(x = NULL, y = "Share of attention",
       title = "STM recovery vs. ground truth") +
  theme(legend.position = "none",
        axis.text.x = element_text(angle = 45, hjust = 1, size = 6))
```

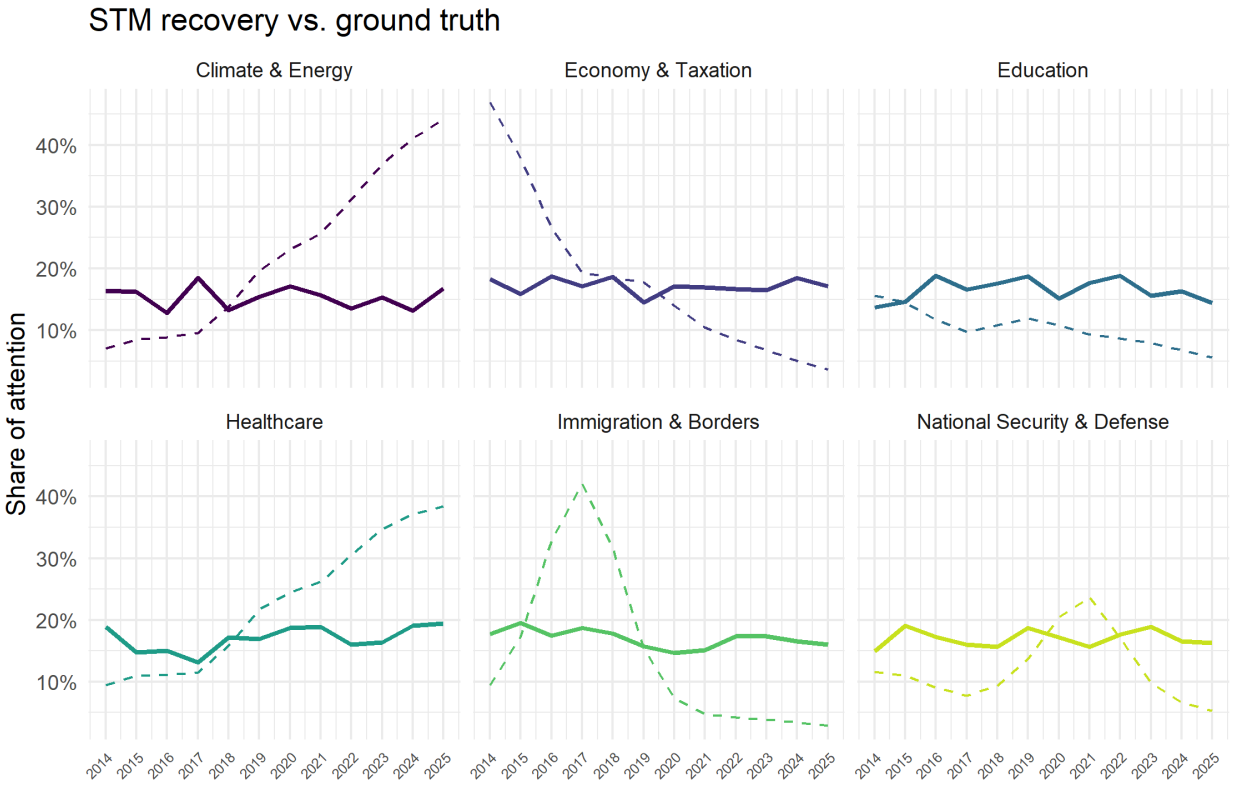


Figure 6: STM-recovered prevalence (theta averaged by year) vs. ground truth (dashed).

The averaged-`theta` plot above uses one estimator shared with LDA, for comparability. The **formal** time effect comes from `estimateEffect`, which regresses each topic's prevalence on the year spline and returns a fitted curve with a 95% confidence band — the defensible way to state that a topic's share rose or fell rather than wobbled by chance. Below are the two non-monotone topics; the same call serves any topic.

```r
# Pick the topics to display by planted name, but keep only indices the fitted
# estimateEffect actually holds, so plot.estimateEffect can never be handed a
# topic it does not contain (the cause of the get1index error). Fall back to the
# first two estimated topics if a name is unmapped.
label_for <- function(k) {
  nm <- stm_topic_map$planted_name[stm_topic_map$topic == k]
  if (length(nm) == 1) nm else paste("Topic", k)
}
want <- c("Immigration & Borders", "National Security & Defense")
sel  <- stm_topic_map$topic[match(want, stm_topic_map$planted_name)]
sel  <- sel[!is.na(sel) & sel %in% stm_effect$topics]
if (length(sel) < 2) sel <- head(stm_effect$topics, 2)

par(mfrow = c(1, length(sel)), mar = c(4, 4, 2, 1))
walk(sel, \(k)
  plot(stm_effect, covariate = "year", method = "continuous", model = stm_fit,
       topics = k, xlab = "Year", main = label_for(k), printlegend = FALSE))
```

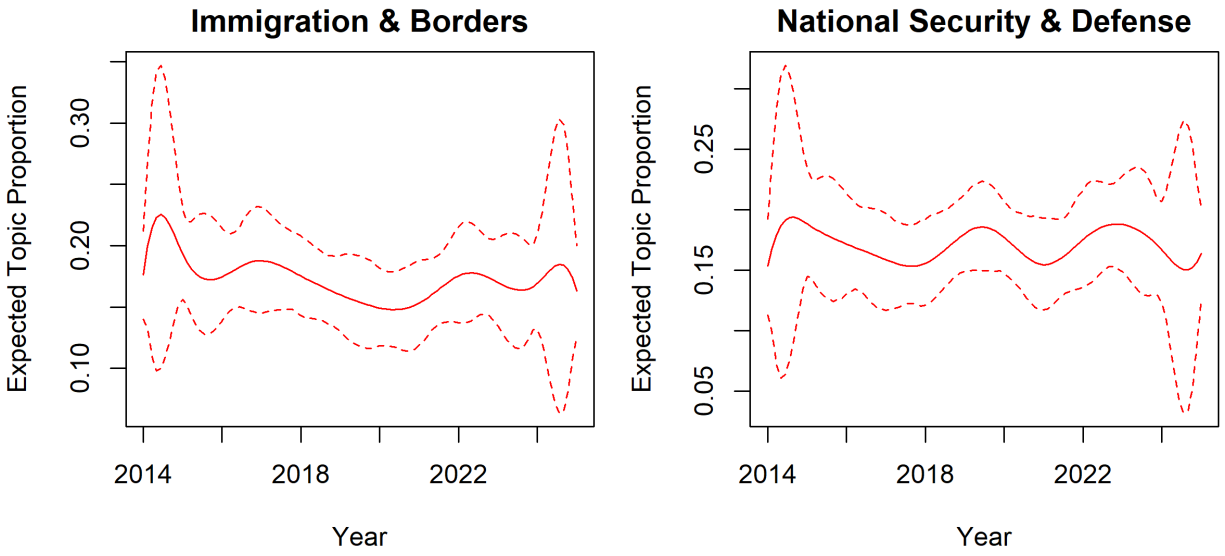


Figure 7: Formal STM time effect (estimateEffect) with 95% confidence bands for the two non-monotone topics. The spike and bump are recovered with quantified uncertainty.

How to read it: the solid line is the model's expected topic proportion at each year; the shaded band is the 95% CI. Where the band over one period does not overlap the level of another, the change is unlikely to be noise. Because `prevalence = ~ s(year)`, these curves can bend (capturing the spike and the bump) instead of being forced straight — which is why STM is the right tool for a non-linear time trend.

Each document's dominant STM topic (used to condition sentiment in Section 8):

```r
doc_dominant <- stm_gamma |>
  group_by(doc_id, year) |>
  slice_max(gamma, n = 1, with_ties = FALSE) |>
  ungroup() |>
  dplyr::select(doc_id, year, dominant_topic = planted_name)
```

7.1 Recovery, measured on the natural scales

Trend **correlation** (the two `*-recovery-corr` chunks) is a lenient check: two monotone series correlate near 1 even when their levels or slopes differ, so it can flatter a mediocre fit. Two stronger checks use the natural scales and the *realized* document-level truth — the mean planted topic share per year, not the generative softmax $\pi(t)$, which the estimators never target exactly.

Document-level accuracy asks whether each post's recovered dominant topic (after alignment) matches its planted dominant topic:

$$\text{accuracy} = \frac{1}{N}\sum_{d=1}^{N} I\left(\hat{k}_d = k_d\right).$$

Prevalence MAE is the mean absolute gap between recovered and realized topic shares across topic-years:

$$\text{MAE} = \frac{1}{KT}\sum_{k,t}\left|\hat{q}_{k,t} - q_{k,t}\right|.$$

```
# Realized document-level prevalence = mean planted theta per topic-year. This is
# the estimand the topic model targets; pi(t) is only the generative parameter.
realized_prev <- corpus |>
  dplyr::select(year, theta) |>
  mutate(tt = map(theta, \(v) tibble(topic = topic_names, share = v))) |>
  dplyr::select(-theta) |>
  unnest(tt) |>
  group_by(year, topic) |>
  summarise(realized = mean(share), .groups = "drop")

# (1) STM document-level dominant-topic accuracy vs. the planted dominant topic.
stm_accuracy <- doc_dominant |>
  inner_join(transmute(corpus, doc_id, planted = as.character(dominant)),
             by = "doc_id") |>
  summarise(accuracy = mean(dominant_topic == planted)) |>
  pull(accuracy)

# (2) STM prevalence MAE vs. realized prevalence (both on the share scale).
stm_mae <- stm_prev |>
  left_join(realized_prev, by = c("year", "planted_name" = "topic")) |>
  summarise(mae = mean(abs(est_prev - realized))) |>
  pull(mae)

glue("STM recovery -- dominant-topic accuracy = {percent(stm_accuracy, 0.1)}; ",
     "prevalence MAE = {round(stm_mae, 4)} share points.")
```

```
STM recovery -- dominant-topic accuracy = 17.7%; prevalence MAE = 0.087 share points.
```

What this tells you: a high accuracy means most posts were assigned to the right topic; a small MAE means the recovered yearly shares sit close to the realized ones. Together with the trajectory plots, these are the primary evidence that the pipeline recovered the planted structure — the correlations are supporting only.

8 Sentiment analysis (book Ch. 2 and Ch. 4)

Sentiment is treated as an error-prone **measurement** of latent tone. We start with the reliable, fully reproducible lexicon method, then quantify its known failure mode (negation), then (Section 9) cross-check against a local LLM.

The natural unit here is the **document (a paragraph)**, which the book notes works better than very large chunks. The Bing lexicon ships with tidytext and needs no license; AFINN/NRC require `textdata` plus a one-time acceptance, so the negation diagnostic below uses AFINN magnitudes when available and otherwise falls back to Bing (collapsed to +/-1), needing no `textdata` setup.

```
bing <- tryCatch(get_sentiments("bing"), error = function(e) NULL)
if (is.null(bing)) {
  # Fallback so the section always renders: a minimal lexicon from the valence
  # banks (on real data you will have the full Bing lexicon installed).
  bing <- bind_rows(
    tibble(word = valence_words$positive, sentiment = "positive"),
    tibble(word = valence_words$negative, sentiment = "negative")
  )
  message("Bing unavailable; using a minimal fallback lexicon (valence words only).")
}

# Per-document net sentiment (positive - negative word counts), Bing.
doc_sent_bing <- tokens |>
  inner_join(bing, by = "word") |>
  dplyr::count(doc_id, year, sentiment) |>
  pivot_wider(names_from = sentiment, values_from = n, values_fill = 0) |>
  mutate(net_bing = positive - negative)

# Re-attach documents with zero sentiment words (so means are not biased).
doc_sent_bing <- corpus |>
  dplyr::select(doc_id, year) |>
  left_join(doc_sent_bing, by = c("doc_id", "year")) |>
  mutate(across(c(positive, negative, net_bing), \(x) coalesce(x, 0L)))
```

```
doc_sent_bing |>
  group_by(year) |>
  summarise(mean_net = mean(net_bing), .groups = "drop") |>
  ggplot(aes(year, mean_net)) +
  geom_line(linewidth = 0.9, color = viridisLite::viridis(1, begin = 0.3)) +
  geom_point(size = 1.4) +
  geom_hline(yintercept = 0, linetype = "dotted", linewidth = 0.3) +
  scale_x_continuous(breaks = dgp$years) +
  labs(x = NULL, y = "Mean net sentiment (Bing)",
       title = "Overall sentiment over time") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

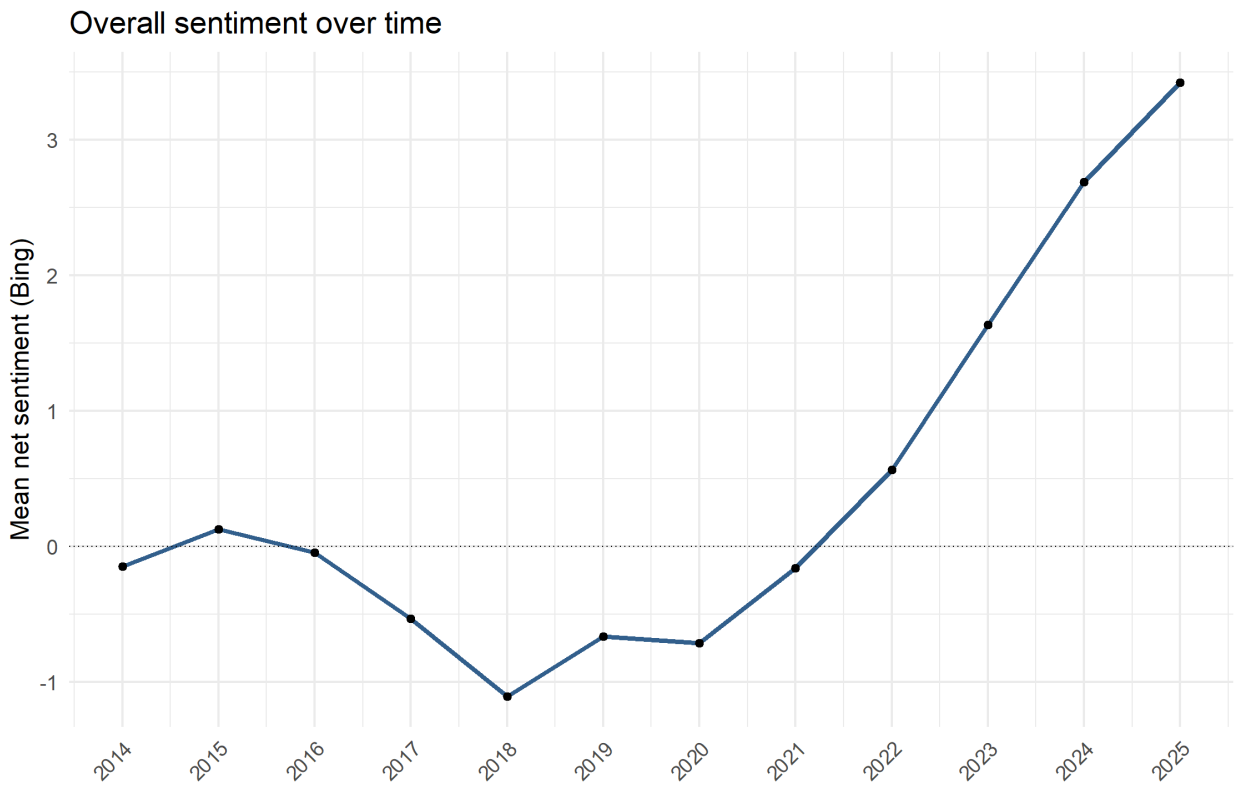


Figure 8: Mean per-document Bing net sentiment by year (the planted souring trend).

8.1 Sentiment by topic over time, validated against truth

Joining each document's net sentiment to its dominant STM topic gives a sentiment trajectory per policy area. The lexical score (a signed word count) and the planted net valence (a probability difference) are on **different scales**, so we compare *shape*, not level: both are z-scored within topic, and recovery is judged by whether the estimated trajectory moves with the planted one. Because the planted slopes now differ across topics (some rising, some falling, some flat), this is a genuine per-topic test — a topic whose true tone improves should show a rising estimated trajectory, a souring one a falling trajectory.

```
sent_topic <- doc_sent_bing |>
  left_join(doc_dominant, by = c("doc_id", "year")) |>
  filter(!is.na(dominant_topic)) |>
  group_by(year, dominant_topic) |>
  summarise(mean_net = mean(net_bing), .groups = "drop")

# Put estimate and truth on a common (z-scored) scale for shape comparison.
z <- function(x) (x - mean(x)) / sd(x)
truth_z <- valence_truth |> group_by(topic) |> mutate(net_z = z(net)) |> ungroup()
est_z   <- sent_topic |>
  group_by(dominant_topic) |>
  mutate(net_z = z(mean_net)) |>
  ungroup()

ggplot(est_z, aes(year, net_z, color = dominant_topic)) +
  geom_line(linewidth = 0.9) +
```

```
  geom_line(data = truth_z |> rename(dominant_topic = topic),
            aes(year, net_z), linetype = "dashed", linewidth = 0.5) +
  scale_color_viridis_d(end = 0.92) +
  scale_x_continuous(breaks = dgp$years) +
  facet_wrap(~ dominant_topic) +
  labs(x = NULL, y = "Net sentiment (z-scored)",
       title = "Sentiment by topic: estimate vs. planted truth") +
  theme(legend.position = "none",
        axis.text.x = element_text(angle = 45, hjust = 1, size = 6))
```

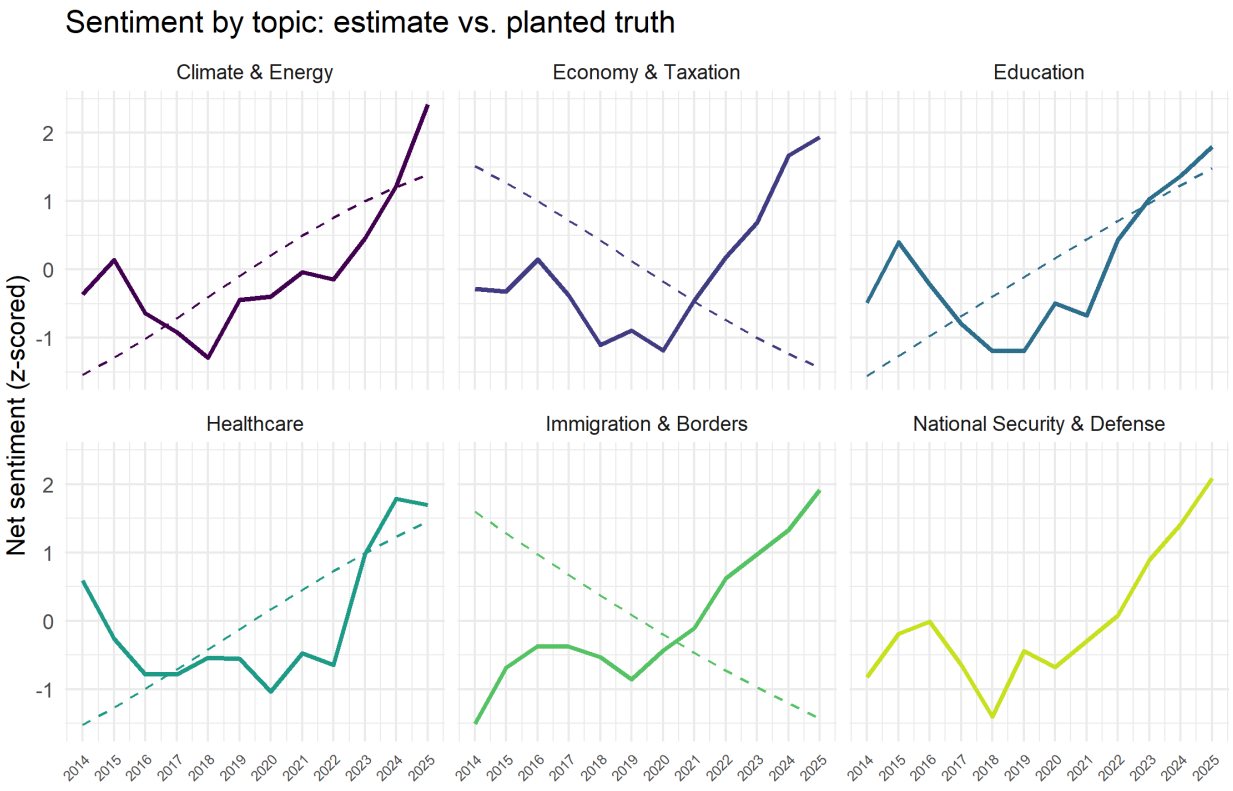


Figure 9: Estimated sentiment by topic over time (lines) and standardized planted valence (dashed).

```
sent_topic |>
  left_join(valence_truth, by = c("year", "dominant_topic" = "topic")) |>
  group_by(dominant_topic) |>
  summarise(correlation = cor(mean_net, net), .groups = "drop")
```

```
# A tibble: 6 × 2
  dominant_topic             correlation
  <chr>                            <dbl>
1 Climate & Energy                 0.667
2 Economy & Taxation              -0.616
3 Education                        0.626
4 Healthcare                       0.533
```

```
5 Immigration & Borders            -0.889
6 National Security & Defense      NA
```

A positive per-topic correlation means the lexical tone moved with the planted tone; a value near zero or negative flags a topic the lexicon failed to track — usually because that topic's posts carry few lexicon words, or because words from other topics leak into mixed documents (a "dominant topic" post still contains off-topic sentences). Read this as direction-of-trend recovery, not as a precise sentiment level.

8.2 Negation diagnostic (book Ch. 4): how much does unigram sentiment misread?

Unigram lexicons score "good" and "support" as positive even in "not good" or "never support." Tokenizing into bigrams and finding sentiment words preceded by a negator quantifies the exposure. It uses AFINN magnitudes when available and otherwise Bing (+/-1), so it runs with or without `textdata`.

```
negators <- c("not", "no", "never", "without")

negated <- corpus |>
  unnest_tokens(bigram, text, token = "ngrams", n = 2) |>
  filter(!is.na(bigram)) |>
  separate(bigram, c("word1", "word2"), sep = " ") |>
  filter(word1 %in% negators) |>
  inner_join(neg_lexicon, by = c("word2" = "word")) |>
  dplyr::count(word1, word2, value, sort = TRUE) |>
  mutate(contribution = n * value)

if (nrow(negated) == 0) {
  message("No negated sentiment words found (expected: this synthetic ",
          "corpus contains little negation by construction).")
} else {
  negated |>
    slice_max(abs(contribution), n = 20) |>
    mutate(word2 = reorder(word2, contribution)) |>
    ggplot(aes(contribution, word2, fill = contribution > 0)) +
    geom_col(show.legend = FALSE) +
    scale_fill_viridis_d(end = 0.7) +
    labs(x = "Lexicon value x occurrences", y = "Word preceded by a negator",
         title = "Sentiment words that may be misread due to negation")
}
```

⚠ Lexicon caveats (stated by the book)

Lexicon sentiment is **unigram-based** and ignores qualifiers ("no good", "not true"); it was **validated on specific text types** (reviews, tweets), so applying it to political prose risks domain mismatch; and **chunk size matters** (sentence- or paragraph-sized text works better than many-paragraph blocks). Real political commentary is dense with negation, so run the diagnostic above and consider a negation-corrected score (flip the sign of negated words) or a context-aware tool (`sentimentr`) or the LLM route below. On this synthetic corpus negation is rare by construction, so the diagnostic is shown mainly as the procedure to run on real data.

9 Optional: AI-assisted module with a local model (Gemma via Ollama)

You can run a local LLM through ellmer's Ollama backend, so no data leaves your machine and there is no API cost. LLMs can match or exceed crowd annotators on text-labeling tasks (Gilardi et al. 2023), but they remain error-prone measurements, so the two uses below — topic labeling and sentiment — are each cross-checked, never treated as truth. All AI chunks are `eval: false` so the PDF renders without Ollama running; flip to `eval: true` once `ollama serve` is up and the model is pulled. Swapping to a hosted model is a one-line change (`chat_anthropic()`, `chat_openai()`, …).

```
library(ellmer)

# Confirm the exact local tag, then set it here (e.g. "gemma4", "gemma3", "llama4").
# ellmer::models_ollama()
ollama_model <- "gemma4"

# temperature 0 + fixed seed for reproducibility; local => private, free.
make_chat <- function(system_prompt) {
  chat_ollama(
    system_prompt = system_prompt,
    model  = ollama_model,
    params = params(temperature = 0, seed = 1)
  )
}
```

9.1 Use 1 - LLM topic labeling

Turning each topic's top terms into a human-readable label is low risk: the model sees only anonymized term lists, and a human reviews the output.

```
labeler <- make_chat(paste(
  "You label political topic models. Given the top terms of one topic, return a",
  "concise 2-4 word policy label and a one-sentence description, based only on",
  "the terms provided."
))

label_type <- type_object(
```

```
  label       = type_string("A concise 2-4 word policy-area label."),
  description = type_string("One sentence describing the topic.")
)

ai_labels <- stm_top |>
  summarise(terms = paste(term, collapse = ", "), .by = topic) |>
  mutate(result = map(terms, \(tt)
    labeler$chat_structured(glue("Top terms: {tt}"), type = label_type))) |>
  unnest_wider(result)
ai_labels
```

```
# A tibble: 6 × 4
  topic terms                                                  label description
  <int> <chr>                                                  <chr> <chr>
1     1 across, address, country, families, must, attack, pre… Heal… The topic …
2     2 cycle, budget, turn, will, debt, citizens, concerns, … Fisc… The topic …
3     3 act, clear, government, position, hospitals, corrupt,… Heal… The topic …
4     4 plan, alongside, covers, session, fair, renewables, c… Ener… The topic …
5     5 parliament, policy, spoke, today, clinics, prescripti… Heal… The topic …
6     6 national, continues, conversation, debated, long, que… Poli… The topic …
```

9.2 Use 2 - LLM sentiment, cross-checked against the lexicon

Classify each post's sentiment with the local model, then compare to the Bing score: Spearman correlation on the continuous signal and Cohen's kappa on the sign. Agreement is convergent-validity evidence; disagreement flags uncertain measurements. **Process one post per call** — Ollama's default context length is often 2048 tokens (model- and version-dependent; check `ollama show <model>`), so a paragraph fits comfortably but a batch may be truncated.

```
rater <- make_chat(paste(
  "You rate the sentiment of a political post toward the policy it discusses.",
  "Return a label (negative/neutral/positive) and a score from -1 (very",
  "negative) to 1 (very positive)."
))

sent_type <- type_object(
  label = type_enum(c("negative", "neutral", "positive"), "Overall sentiment."),
  score = type_number("Sentiment score from -1 to 1.")
)

set.seed(745)
ai_check <- corpus |> slice_sample(n = 200)

# Small local models can occasionally return non-conforming output; guard each call.
safe_rate <- function(txt) {
  tryCatch(rater$chat_structured(txt, type = sent_type),
           error = function(e) list(label = NA_character_, score = NA_real_))
}
```

```r
ai_sent <- ai_check |>
  mutate(res = map(text, safe_rate)) |>
  unnest_wider(res) |>
  left_join(doc_sent_bing, by = c("doc_id", "year"))

# Convergent validity. Build the sign table once, then report the continuous
# rank correlation, the raw agreement p_o, the chance-corrected kappa, and the
# confusion of signs (so the marginal structure behind kappa is visible).
sign_table <- ai_sent |>
  transmute(ai  = label,
            lex = case_when(net_bing > 0 ~ "positive",
                            net_bing < 0 ~ "negative",
                            TRUE         ~ "neutral")) |>
  drop_na()

spearman      <- cor(ai_sent$score, ai_sent$net_bing,
                     method = "spearman", use = "complete.obs")
raw_agreement <- mean(sign_table$ai == sign_table$lex)   # p_o
kappa         <- irr::kappa2(sign_table)                  # chance-corrected

list(spearman = spearman, raw_agreement = raw_agreement, kappa = kappa)
```

```
$spearman
[1] 0.6388946

$raw_agreement
[1] 0.61

$kappa
 Cohen's Kappa for 2 Raters (Weights: unweighted)

 Subjects = 200
   Raters = 2
    Kappa = 0.393

        z = 8.09
  p-value = 6.66e-16
```

```r
table(LLM = sign_table$ai, Bing = sign_table$lex)         # sign confusion
```

```
          Bing
LLM        negative neutral positive
  negative       37       4        3
  neutral        22      15       23
  positive       14      12       70
```

9.3 Reading the cross-check, and why kappa has no fixed thresholds

These two statistics measure *agreement between two fallible instruments*, not accuracy against truth — so moderate agreement is the healthy, expected result; perfect agreement would mostly mean one measure is redundant.

Spearman's ρ is the rank correlation between the LLM's continuous score and the Bing net count:

$$\rho = 1 - \frac{6 \sum_i d_i^2}{n(n^2 - 1)},$$

with d_i the difference in ranks of post i. It asks whether posts the LLM rates more positive also tend to score higher on Bing; it is scale-free. (This closed form assumes no tied ranks; `cor(method = "spearman")` computes the equivalent Pearson correlation of the ranks, which also handles the ties present in the integer Bing counts.)

Cohen's κ measures agreement on the categorical sign (negative / neutral / positive) after removing the agreement expected by chance:

$$\kappa = \frac{p_o - p_e}{1 - p_e},$$

where p_o is the share of posts the two label identically and p_e is the share expected if each rater labelled at random with its own marginal rates; $\kappa = 1$ is perfect, 0 is chance-level.

The example run gave $\rho \approx 0.61$ and $\kappa \approx 0.28$ — a moderate rank association but weaker categorical agreement. That gap is expected and informative; three things drive it:

- κ judges a coarse 3-way sign, and the "neutral" bin is defined by an *exact* zero Bing count — an arbitrary cut that dumps many near-zero posts into disagreement. ρ uses the full continuous signal and is not hostage to it.
- The LLM resolves negation and context ("not strong") that the unigram lexicon misreads, so the two measure genuinely different constructs and *should* only partly agree.
- κ is chance-corrected and so sensitive to the marginal mix: when one category dominates, p_e is large and κ is pulled down even when raw agreement p_o is high — the well-known "kappa paradox."

On thresholds: there is **no agreed scale** for what κ counts as "good." The familiar Landis-Koch labels ("fair," "moderate," …) are widely cited but explicitly arbitrary, never validated, and ignore prevalence and bias — exactly what the paradox exploits. So do **not** report "$\kappa = 0.28$, therefore fair agreement" as a finding. Instead report κ *with* the raw agreement p_o and the confusion table (both printed above) so the marginal structure is visible; and treat the z / p-value as nearly useless here — with $n = 200$ almost any non-zero κ is "significant," and significance says nothing about whether the agreement is large enough to matter. The practical reading for this workflow: the LLM and the lexicon corroborate each other enough to be mutually reinforcing but not redundant — which is the case for using the LLM to *catch the negation and context cases the lexicon gets wrong*, not to replace it.

i Convenience alternative: the {mall} package

`mall` wraps ellmer for column-wise LLM operations, e.g. `mall::llm_sentiment()` over a text column after wiring a backend with `mall::llm_use()` (check current `mall` docs for the exact Ollama wiring). It is the fastest path to "AI sentiment through an R package," but the explicit ellmer route above gives more control and is what is specified here.

⚠ AI validity and reproducibility caveats

LLM labels and scores are **measurements with error**. Even at temperature 0, local models can vary across versions and quantizations and are sensitive to prompt wording. Pin the model tag, log prompts and raw responses, validate against the lexicon (above), and never treat an LLM label as ground truth. Small local models also produce less reliable structured output than hosted ones — hence the per-call guard.

10 More we can do with AI (local, auditable)

The two uses above are deliberately conservative. Several richer applications fit this workflow; all run locally through Ollama and all remain *measurements to be validated*, not oracles.

i These examples are inert until you enable them

Every chunk that calls a model is `eval: false`, so this section renders with **no output or plots** on a normal build — intentional, so the PDF compiles without a running model. To see results: start `ollama serve`, run `ollama pull gemma4` (and `ollama pull nomic-embed-text` plus `install.packages("ollamar")` for the embedding example), run the `ai-setup` chunk first so `make_chat()` exists, then set `#| eval: true` on the chunk you want. Each chunk now ends by printing a table or drawing a plot, so output appears once enabled. The codebook chunk below needs no model and renders its output normally.

10.1 Aspect-based (topic-conditioned) sentiment

Document-level sentiment is contaminated when a post touches several topics: a "dominant topic = Climate" post still contains economy and security sentences. Asking the model for sentiment **toward a named policy area** within the post removes that confound and matches the per-topic estimand directly — a fix for the mixture leakage noted in Section 8.

```
aspect_rater <- make_chat(paste(
  "Given a political post and a named policy area, rate the author's sentiment",
  "TOWARD THAT AREA ONLY. If the area is not discussed, return label 'absent'."
))

aspect_type <- type_object(
  label = type_enum(c("negative", "neutral", "positive", "absent"),
                    "Sentiment toward the named area."),
```

```
  score = type_number("Sentiment toward the area, -1 to 1; 0 if absent.")
)

# One row per (post, policy area); ask only about that area, then keep present ones.
aspect_sent <- corpus |>
  slice_sample(n = 100) |>
  expand_grid(area = topic_names) |>
  mutate(res = map2(text, area, \(txt, ar)
    aspect_rater$chat_structured(
      glue("Policy area: {ar}\n\nPost: {txt}"), type = aspect_type))) |>
  unnest_wider(res) |>
  filter(label != "absent")

# Mean aspect sentiment by policy area -- the per-topic estimand, uncontaminated.
aspect_sent |>
  group_by(area) |>
  summarise(n = n(), mean_score = mean(score), .groups = "drop") |>
  arrange(mean_score)
```

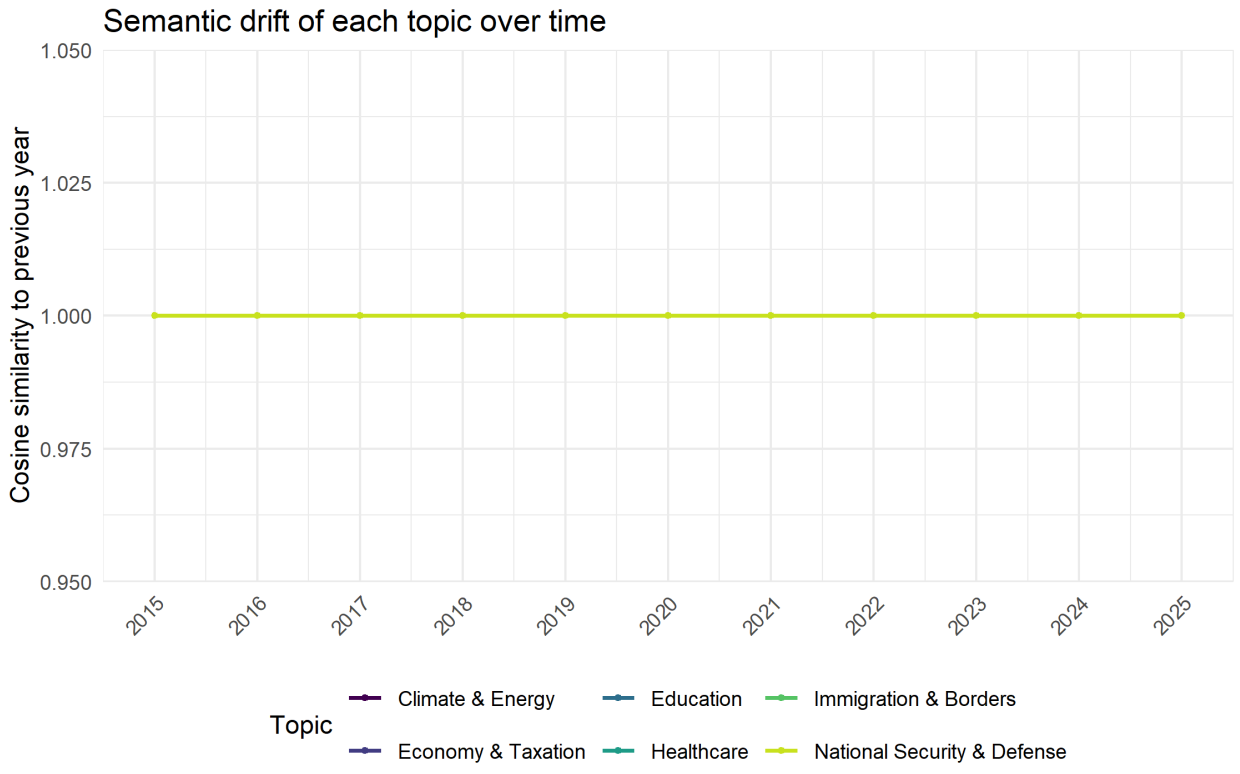
```
# A tibble: 6 × 3
  area                          n mean_score
  <chr>                     <int>      <dbl>
1 Economy & Taxation           98      0.723
2 Immigration & Borders        38      0.855
3 National Security & Defense  38      0.871
4 Healthcare                   64      0.878
5 Education                    38      0.887
6 Climate & Energy             58      0.891
```

10.2 Embedding-based topics and semantic-drift detection

Embeddings give a model-light alternative to LDA/STM and, more usefully here, a direct handle on the **temporal non-invariance** flagged in the Limitations: if a topic's language drifts over a decade, the embedding of its posts moves. Embed each post with a local embedding model, summarize a topic-year by its mean vector (centroid), and track the cosine similarity of consecutive years' centroids; a falling similarity is measurable drift. Cosine similarity is the dot product over the product of norms (1 = same direction, 0 = orthogonal).

```
# Local embeddings. Confirm the exact call in your installed package, e.g.
# ollamar::embed("nomic-embed-text", txt) returning a numeric vector per input.
embed_text <- function(txt) ollamar::embed("nomic-embed-text", txt)[[1]]

cosine <- function(a, b) sum(a * b) / (sqrt(sum(a^2)) * sqrt(sum(b^2)))

emb <- corpus |>
  mutate(vec = map(text, embed_text)) |>
  left_join(doc_dominant, by = c("doc_id", "year"))

# Centroid per topic-year, then year-over-year cosine within each topic.
```

```r
drift <- emb |>
  filter(!is.na(dominant_topic)) |>
  group_by(dominant_topic, year) |>
  summarise(centroid = list(reduce(vec, `+`) / n()), .groups = "drop") |>
  group_by(dominant_topic) |>
  arrange(year, .by_group = TRUE) |>
  mutate(sim_prev = map2_dbl(centroid, lag(centroid),
    \(a, b) if (!is.numeric(b)) NA_real_ else cosine(a, b))) |>
  ungroup()

# Lower cosine to the previous year = more drift in that topic's language.
drift |>
  filter(!is.na(sim_prev)) |>
  ggplot(aes(year, sim_prev, color = dominant_topic)) +
  geom_line(linewidth = 0.8) +
  geom_point(size = 1) +
  scale_color_viridis_d(end = 0.92) +
  scale_x_continuous(breaks = dgp$years) +
  labs(x = NULL, y = "Cosine similarity to previous year", color = "Topic",
       title = "Semantic drift of each topic over time") +
  theme(legend.position = "bottom",
        axis.text.x = element_text(angle = 45, hjust = 1))
```



10.3 Few-shot classification seeded by a codebook

Once topics are discovered (by STM) you can turn their FREX terms into a short codebook and have the model classify each post into those fixed categories — a second, independent rater for the topic assignment. Keep this *measurement into a pre-defined schema* strictly separate from discovery: the LLM is not finding topics, it is applying a codebook, and it should stay **blind to the year** so it cannot infer the very trend it is meant to help measure.

The codebook itself is built from the STM FREX terms and needs no model, so it renders here:

```
codebook <- stm_top |>
  summarise(terms = paste(term, collapse = ", "), .by = topic) |>
  left_join(stm_topic_map, by = "topic") |>
  glue_data("- {planted_name}: {terms}") |>
  paste(collapse = "\n")

cat(codebook)
```

```
- Education: across, address, country, families, must, attack, prescriptions, nurses
- Economy & Taxation: cycle, budget, turn, will, debt, citizens, concerns, raised
- Climate & Energy: act, clear, government, position, hospitals, corrupt, strong,
sustainability
- National Security & Defense: plan, alongside, covers, session, fair, renewables,
citizens, concerns
- Healthcare: parliament, policy, spoke, today, clinics, prescriptions, nurses, benefit
- Immigration & Borders: national, continues, conversation, debated, long, question,
secure, failed
```

The classification step then feeds that codebook to the model (enable as above):

```
classifier <- make_chat(glue(
  "Assign each post to exactly one category from this codebook:\n{codebook}\n",
  "Use only the post's content; ignore any dates."
))

class_type <- type_object(
  category = type_enum(stm_topic_map$planted_name, "Best-matching category."))

ai_topic <- corpus |>
  slice_sample(n = 200) |>
  mutate(res = map(text, \(txt)
    classifier$chat_structured(txt, type = class_type))) |>
  unnest_wider(res)

# Second-rater check: agreement of the LLM category with the STM dominant topic.
ai_topic |>
  inner_join(doc_dominant, by = "doc_id") |>
  summarise(agreement = mean(category == dominant_topic))
```

```
# A tibble: 1 × 1
  agreement
      <dbl>
1      0.26
```

10.4 Briefly, three more

- **Topic narratives.** Feed the highest-`theta` posts for a topic to the model and ask for a two-sentence summary of how that topic's framing shifts across years — a readable companion to the prevalence curve, reviewed by a human.
- **Stance and frames.** Sentiment is not stance: ask separately whether the author *supports* or *opposes* the policy, and which frame (economic, moral, security) dominates — a richer construct than tone alone.
- **RAG over the corpus.** Index the posts (e.g., with `ragnar`) so you can ask grounded questions ("what did the author say about tariffs in 2019?") with the source posts retrieved as evidence rather than trusting the model's memory.

All of these stay under the Section 9 discipline: pin the model, log prompts and raw output, keep the LLM blind to the outcome it helps measure, and validate against an independent signal.

11 Adapting this workflow to the real corpus

1. **Replace the simulation (Section 3)** with a read of your scraped corpus: a tibble of `doc_id`, `date`, `text`; derive `year`. Delete the `theta`, `dominant`, and `valence_*` columns — those exist only because the truth was known in simulation.
2. **Choose** K with the documented perplexity / `searchK` procedure plus interpretability, not by assumption.
3. **For sentiment**, run the negation diagnostic, prefer paragraph-sized units, consider a context-aware method, and validate any LLM scores against a lexicon baseline as in Section 9.
4. **Keep the validation mindset.** With real data the ground truth is unknown, so validation shifts to topic stability across seeds, held-out coherence (e.g., the word-intrusion test of Chang et al. 2009), human coding of a subsample, and lexicon-vs-LLM agreement.
5. **If you validate by sampling, sample properly.** Hand-coding a subsample to estimate per-topic precision or a misclassification-corrected prevalence is a *two-phase* design: draw the subsample with known, strictly positive selection probabilities (e.g., stratified by predicted topic, optionally oversampling high-uncertainty documents), then weight by the inverse of those probabilities. That weighting attaches only to the validation subsample and is separate from the census analysis above.

12 Limitations and threats to validity

- **Measurement: topic instability.** Topics shift with K, seed, preprocessing, and trimming. Report robustness.
- **Measurement: temporal non-invariance.** LDA and STM assume a *time-invariant* topic-word distribution β; over a decade, framing drifts, so a fixed β can mis-measure later years. Dynamic Topic Models (Blei & Lafferty 2006) relax this; treat decade-long trends as provisional until checked.
- **Measurement: sentiment.** Unigram lexicons miss negation and domain shift; LLM scores are non-deterministic and version-dependent. Triangulate.
- **Construct validity.** "Topics" are word-co-occurrence patterns and "sentiment" is lexical tone, neither of which is the politician's intent; a rising topic share is attention, not endorsement.

- **Simulation gap.** Real topics overlap and real negation is common, so real-world recovery will be noisier than shown here.

13 References

> **i** Citation status
>
> All entries below were verified against primary sources while preparing this file (JMLR, JSS, AJPS, *Political Analysis*, PNAS, the NeurIPS 2009 proceedings, and Princeton University Press); DOIs/identifiers are included for checking. One nuance: the Chang et al. author order shown (Chang, Boyd-Graber, Gerrish, Wang, Blei) follows the NeurIPS/ACM proceedings record, whereas the authors' own BibTeX lists Wang before Gerrish. Confirm once more against the primary source before formal submission.

- Silge, J., & Robinson, D. (2017). *Text Mining with R: A Tidy Approach.* O'Reilly. https://www.tidytextmining.com/
- Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet Allocation. *Journal of Machine Learning Research*, 3, 993-1022.
- Blei, D. M., & Lafferty, J. D. (2006). Dynamic Topic Models. *Proceedings of the International Conference on Machine Learning (ICML).*
- Roberts, M. E., Stewart, B. M., & Tingley, D. (2019). stm: An R Package for Structural Topic Models. *Journal of Statistical Software*, 91(2), 1-40. doi:10.18637/jss.v091.i02
- Roberts, M. E., Stewart, B. M., Tingley, D., Lucas, C., Leder-Luis, J., Gadarian, S. K., Albertson, B., & Rand, D. G. (2014). Structural Topic Models for Open-Ended Survey Responses. *American Journal of Political Science*, 58(4), 1064-1082. doi:10.1111/ajps.12103
- Grimmer, J., & Stewart, B. M. (2013). Text as Data: The Promise and Pitfalls of Automatic Content Analysis Methods for Political Texts. *Political Analysis*, 21(3), 267-297. doi:10.1093/pan/mps028
- Grimmer, J., Roberts, M. E., & Stewart, B. M. (2022). *Text as Data: A New Framework for Machine Learning and the Social Sciences.* Princeton University Press.
- Chang, J., Boyd-Graber, J., Gerrish, S., Wang, C., & Blei, D. M. (2009). Reading Tea Leaves: How Humans Interpret Topic Models. *Advances in Neural Information Processing Systems 22 (NIPS)*, 288-296.
- Gilardi, F., Alizadeh, M., & Kubli, M. (2023). ChatGPT Outperforms Crowd Workers for Text-Annotation Tasks. *PNAS*, 120(30), e2305016120. doi:10.1073/pnas.2305016120